

Tutorial 1: Basic Programming Tasks of OCaml

Programming Languages & Compilers
School of Computer Science, Wuhan University

Spring 2025

目录

1	开发环境配置	2
1.1	选择配置方式	2
1.2	配置步骤	2
1.2.1	OCaml	2
1.2.2	Visual Studio Code (VS Code)	3
2	运行你的第一个 OCaml 程序	4
2.1	使用 OCaml 交互式解释器 (REPL)	4
2.2	编写独立的 OCaml 源文件	4
3	OCaml 基础知识	5
3.1	基本数据类型	5
3.2	算术和逻辑运算	5
3.3	列表	6
3.4	if-else 条件分支	6
3.5	函数定义	6
3.6	函数作为值	7

3.7 模式匹配	8
4 编程练习	9
4.1 练习 1: 实现 Fibonacci 函数	9
4.2 练习 2: 计算列表的累乘	9
4.3 ※ 练习 3: 实现函数组合	10
4.4 ※ 练习 4: 实现 <code>for_all</code> 函数	10
5 参考资源	11

1 开发环境配置

1.1 选择配置方式

1. 原生安装方案: 如果你使用的是 Linux/macOS 系统, 在本机系统上直接安装 OCaml 环境即可。Windows 系统也可以原生安装 OCaml 环境, 但不推荐, 需要的同学可以参考 <https://ocaml.org/install#windows>。
2. WSL 方案: 如果你使用的是 Windows 系统, 可以选择 WSL (Windows Subsystem for Linux)。启用 WSL 功能并安装 Linux 发行版后, 按照 Linux 配置步骤进行配置。
3. 虚拟机方案: 除了 WSL, 你也可以选择安装一个 Linux 虚拟机。我们提供了一个预配置的 Ubuntu Linux 虚拟机 (OVA 格式, 可直接导入 VMware 使用), 有需要的同学可以找助教拷贝或从群文件下载虚拟机, 免去后面的配置步骤。

1.2 配置步骤

如果你已经安装好了预配置虚拟机, 可以直接跳过本小节。

以下配置步骤适用于 Linux (包括 WSL) 或 macOS 系统。本教程将以 Debian/Ubuntu 为例说明 Linux 系统下的配置步骤, 如果你使用其他 Linux 发行版, 请根据实际情况调整包管理器命令。

1.2.1 OCaml

安装和初始化 opam 包管理器 opam 用于安装和管理 OCaml 的各种工具和库。

打开终端并运行以下命令安装 `opam`:

- Linux: `sudo apt install opam`
- macOS: `brew install opam`

然后运行以下命令初始化 `opam`:

```
opam init
```

注意

初始化过程中，系统会询问你是否配置环境变量，建议选择 `y`，将 `opam` 环境配置添加到 `~/.profile` 中（这样可以在任何位置使用 `opam` 环境）。否则，每次需要 `opam` 环境时，都必须手动运行 `eval $(opam env)` 来激活。

最后运行以下命令激活 `opam` 环境:

```
eval $(opam env --switch=default)
```

安装 OCaml 开发环境 配置好 `opam` 后，我们还需要安装一些 OCaml 开发必备的工具:

- `ocaml-lsp-server`: 语言服务器，提供代码补全等功能;
- `odoc`: 文档生成工具;
- `ocamlformat`: 代码格式化工具;
- `utop`: 增强版的 OCaml 交互式环境。

在终端运行以下命令安装它们:

```
opam install ocaml-lsp-server odoc ocamlformat utop
```

1.2.2 Visual Studio Code (VS Code)

注意

如果你使用的是 WSL，VS Code 需要安装在 Windows 侧，然后安装 WSL 插件。

1. 从官网下载并安装 VS Code: <https://code.visualstudio.com/>
2. 打开 VS Code，点击左侧扩展图标（或按 `Ctrl+Shift+X`），搜索并安装 *OCaml Platform*（OCaml 语言服务器）和 *ocamlformat-vscode*（OCaml 代码格式化工具）插件。

2 运行你的第一个 OCaml 程序

2.1 使用 OCaml 交互式解释器 (REPL)

REPL (Read-Eval-Print Loop) 是 OCaml 提供的交互式环境，顾名思义，它读取用户输入的代码，求值，输出结果，然后继续接受输入，循环往复。

在终端中运行 `ocaml` 命令启动 REPL。在 `#` 提示符后输入表达式，用双分号 (`;;`) 表示表达式的结束，然后按回车键求值。表达式可以是多行的。输入 `Ctrl+D` (Unix) 或 `Ctrl+Z` (Windows) 退出 REPL。

示例：

```
1  # let greeting = "Hello, OCaml!";;          (* 定义一个字符串 *)
2  val greeting : string = "Hello, OCaml!"      (* REPL 输出 *)
3  # print_endline greeting;;                  (* 打印字符串并换行 *)
4  Hello, OCaml!                                (* REPL 输出 *)
5  # ^Z                                         (* 退出 REPL *)
6  [1]+  Stopped                               ocaml
```

2.2 编写独立的 OCaml 源文件

打开终端并运行以下命令，新建一个 VS Code 工作区并进入：

```
1  mkdir ocaml      # 创建一个目录
2  cd ocaml         # 进入目录
3  code .           # 用 VS Code 打开当前目录
```

注意

在 WSL 终端中，`code .` 命令会启动 Windows 侧的 VS Code。还有一种等价的方式：在 Windows 终端中，运行 `wsl code .` 命令，VS Code 将通过 WSL 的文件系统打开当前目录。

在 VS Code 中创建并编辑源代码文件（扩展名为 `.ml`）：

代码 1: hello.ml

```
1  let greeting = "Hello, OCaml!"      (* 定义一个字符串 *)
2  let () =                             (* 程序入口，多条语句用分号隔开 *)
3      print_string greeting;          (* 打印字符串 *)
4      print_newline ()                (* 换行 *)
```

保存文件，按 `Ctrl+`` 打开 VS Code 内置终端，编译并运行：

```
1  ocamlc -o hello hello.ml    # 编译 hello.ml, 生成可执行文件 hello
2  ./hello                    # 运行可执行文件
```

3 OCaml 基础知识

3.1 基本数据类型

```
1  let x = 42                  (* 整数 int *)
2  let y = 3.14                (* 浮点数 float *)
3  let z = "Hello!"           (* 字符串 string *)
4  let b = true                (* 布尔值 bool *)
5  let c = 'a'                 (* 字符 char *)
```

注意

OCaml 是静态类型语言，但编译器会自动推导类型，所以通常不需要显式声明类型。

3.2 算术和逻辑运算

```
1  let sum = 3 + 5             (* 整数加法 *)
2  let diff = 7 - 2            (* 整数减法 *)
3  let prod = 2 * 4            (* 整数乘法 *)
4  let quot = 10 / 3           (* 整数除法 *)
5  let rem = 10 mod 3          (* 整数求余 *)
6
7  let fsum = 3.14 + 2.71      (* 浮点加法 *)
8  let fdiff = 3.14 - 2.71     (* 浮点减法 *)
9  let fprod = 3.14 * 2.71     (* 浮点乘法 *)
10 let fquot = 3.14 /. 2.71     (* 浮点除法 *)
11 let fpow = 2.0 ** 3.0       (* 浮点乘方 *)
12
13 let eq = 3 = 3               (* 等于 *)
14 let neq = 3 <> 4             (* 不等于 *)
15 let gt = 5 > 3               (* 大于 *)
16 let lt = 2 < 7               (* 小于 *)
17 let ge = 5 >= 5              (* 大于等于 *)
18 let le = 3 <= 2              (* 小于等于 *)
19
20 let and_op = true && false    (* 逻辑与 *)
21 let or_op = true || false    (* 逻辑或 *)
22 let not_op = not true        (* 逻辑非 *)
```

3.3 列表

OCaml 中的列表是一种不可变的数据结构，可以存储任意类型的元素：

```
1  let lst = [1; 2; 3; 4]           (* 创建列表 *)
2  let lst' = 1 :: 2 :: 3 :: []    (* 使用 :: 构造列表 *)
3  let lst'' = [1; 2] @ [3; 4]    (* 使用 @ 连接列表 *)
```

其中，`::` 运算符将左侧的元素添加到右侧的列表中，得到一个新列表；它是右结合的，所以 `1 :: 2 :: 3 :: []` 相当于 `1 :: (2 :: (3 :: []))`。

拓展

可以使用 `List` 模块中的函数来操作列表，例如：

```
1  let len = List.length lst        (* 计算列表长度 *)
2  let head = List.hd lst           (* 获取列表的第一个元素 *)
3  let tail = List.tl lst           (* 获取除第一个元素外的子列表 *)
4  let rev = List.rev lst           (* 反转列表 *)
5  let sorted = List.sort compare lst (* 排序列表 *)
```

3.4 if-else 条件分支

`if-else` 是 OCaml 中的条件分支结构，用于根据条件选择不同的表达式：

```
1  let n = 42
2  let is_even =
3      if n mod 2 = 0 then true    (* n 是偶数 *)
4      else false                 (* n 是奇数 *)
```

注意

`if-else` 是表达式，因此必须有 `else` 分支；如果没有 `else` 分支，则需要显式返回 `()` (OCaml 的空值)。

3.5 函数定义

OCaml 中有两种等价的函数定义方式：

```
1  let add a b = a + b              (* 方式 1: 直接定义 *)
2  let add' = fun a b -> a + b      (* 方式 2: 使用 fun 关键字 *)
3
4  let result = add 3 5              (* 返回 8 *)
5  let result' = add (add 1 2) (add' 3 4) (* 返回 10 *)
```

其中, `add (add 1 2) (add' 3 4)` 相当于 $(1+2)+(3+4)$, 即: 先计算表达式 `add 1 2` 和 `add' 3 4`, 再计算它们的返回值的和。

注意

嵌套函数调用时, 内层函数调用需要用括号包裹, 括号内函数调用的返回值作为外层函数的参数。

使用 `rec` 关键字定义递归函数:

```
1  let rec fact n =                (* 计算 n 的阶乘 *)
2      if n = 0 then 1             (* 递归终止条件: 0! = 1 *)
3      else n * fact (n - 1)       (* 递归调用自身: n! = n(n-1)! *)
4
5  let result = fact 5             (* 返回 120 *)
```

拓展

你可能发现了, 当 $n < 0$ 时, `fact` 函数会陷入无限递归。为了避免这种情况, 我们可以在非法输入时用 `failwith` 关键字抛出异常:

```
1  let rec fact n =
2      if n < 0 then failwith "n should be non-negative"
3      else if n = 0 then 1
4      else n * fact (n - 1)
5
6  let result = fact 5             (* 返回 120 *)
7  let _ = fact (-1)              (* 抛出异常 *)
```

3.6 函数作为值

在 OCaml 中, 函数是“一等公民”(first-class citizens), 可以作为值传递给其他函数、作为函数的返回值, 还可以存储在变量或数据结构中。

例如:

```
1  let double x = x * 2           (* 定义一个函数 *)
2  let f = double                 (* 将函数存储在变量中 *)
3  let result = f 21              (* 返回 42 *)
4
5  let apply_twice f x = f (f x)  (* 函数作为参数 *)
6  let result' = apply_twice double 10 (* 返回 40: 10 * 2 * 2 *)
7
8  let get_op = function         (* 函数作为返回值 *)
9      | "+" -> ( + )            (* 返回加法运算符 *)
10     | "*" -> ( * )            (* 返回乘法运算符 *)
```

```

11     | _ -> failwith "Unknown operator"
12
13     let add = get_op "+"           (* 获取加法函数 *)
14     let result = add 3 5          (* 返回 8 *)

```

注意

运算符实际上也是函数。例如，`( + )` 是一个接受两个参数的函数，等价于 `fun x y → x + y`。

拓展

高阶函数 (higher-order functions) 是指接受函数作为参数或返回函数的函数。例如，在上面的代码中，`apply_twice` 和 `get_op` 就是高阶函数。

3.7 模式匹配

模式匹配是 OCaml 中处理分支逻辑的另一种方式：

```

1  let n = 42
2  let describe_number =
3      match n with           (* 匹配 n 的值 *)
4      | 0 -> "Zero"          (* n 为 0 返回 "Zero" *)
5      | 1 -> "One"           (* n 为 1 返回 "One" *)
6      | _ -> "Other"         (* 其他情况返回 "Other" *)

```

其中 `_` 是通配符模式，匹配任何值。

模式匹配也可以用于处理列表：

```

1  let rec sum lst =
2      match lst with
3      | [] -> 0               (* 空列表返回 0 *)
4      | head :: tail -> head + sum tail  (* 递归求和 *)
5
6  let result = sum [1; 2; 3; 4; 5]      (* 返回 15 *)

```

其中 `head :: tail` 是列表解构的模式，表示将列表分解为第一个元素 (`head`) 和剩余元素列表 (`tail`)。

拓展

函数定义可以使用 `function` 关键字，简化模式匹配的写法：

```

1  let rec sum = function           (* 相当于 fun 和 match 的组合 *)
2      | [] -> 0                      (* 空列表返回 0 *)
3      | head :: tail -> head + sum tail (* 递归求和 *)
4
5  let result = sum [1; 2; 3; 4; 5]   (* 返回 15 *)

```

4 编程练习

本节包含基础练习和拓展练习（标记为 * 的练习题），请在下课前完成。基础练习需要掌握，拓展练习供有余力的同学选做。

4.1 练习 1：实现 Fibonacci 函数

编写一个递归函数 `fib`，计算第 n 个 Fibonacci 数 F_n 。

其中

$$F_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F_{n-1} + F_{n-2} & \text{otherwise} \end{cases}, n \in \mathbb{N}$$

调用示例：

```

1  let result = fib 19   (* 返回 4181 *)

```

要求：

1. 函数签名: `val fib : int → int`;
2. 对于非法输入 ($n < 0$) 的情况要适当处理。

4.2 练习 2：计算列表的累乘

编写一个递归函数 `prod`，计算列表 $[x_1; x_2; \dots; x_n]$ 的累乘 $\prod_{i=1}^n x_i$ 。

调用示例：

```

1  let result = prod [1; 2; 3; 4; 5]   (* 返回 120 *)

```

要求:

1. 函数签名: `val prod : int list → int;`
2. 使用模式匹配处理列表。

4.3 ※ 练习 3: 实现函数组合

编写一个函数 `compose`, 实现两个函数的组合运算。对于函数 f 和 g , 它们的组合 $f \circ g$ 定义为: $(f \circ g)(x) = f(g(x))$ 。

调用示例:

```

1  let double x = x * 2           (* 将数字翻倍 *)
2  let add_one x = x + 1         (* 将数字加一 *)
3
4  let f = compose double add_one (* 组合函数: 先加一再翻倍 *)
5  let result = f 3              (* 返回 8: (3 + 1) * 2 *)
```

要求:

1. 函数签名: `val compose : ('a → 'b) → ('c → 'a) → 'c → 'b;`

注意

'a、'b、'c 等是类型变量 (type variables), 用于表示通用类型。例如, 'a 表示任意类型, 'a list 表示任意类型的列表。

4.4 ※ 练习 4: 实现 `for_all` 函数

编写一个函数 `for_all`, 传入一个函数 $p: 'a \rightarrow \text{bool}$ 和一个列表 `lst`, 检查列表中的所有元素是否都满足 p 指定的条件。

调用示例:

```

1  let is_positive x = x > 0      (* 判断是否为正数 *)
2  let is_even x = x mod 2 = 0   (* 判断是否为偶数 *)
3
4  let r1 = for_all is_positive [1; 2; 3] (* 返回 true *)
5  let r2 = for_all is_even [2; 4; 7]    (* 返回 false *)
6  let r3 = for_all is_positive []       (* 返回 true *)
```

要求:

1. 函数签名: `val for_all : ('a → bool) → 'a list → bool;`
2. 空列表应返回 `true`。

5 参考资源

1. OCaml 官方教程: <https://ocaml.org/docs/get-started>
2. OCaml 官方文档: <https://ocaml.org/docs/language>